



UNIVERSITÀ
DI TRENTO

Dipartimento di
Ingegneria e Scienza dell'Informazione

Dipartimento di Ingegneria e Scienza dell'Informazione

Corso di Laurea in
Ingegneria Informatica, delle Comunicazioni ed Elettronica

CORSO DI PROGRAMMAZIONE AVANZATA

Linguaggio C++ 11

Anno accademico 2023/2024

1 UML

2 Ridefinizione degli operatori

Il linguaggio C++ supporta la ridefinizione degli operatori. Gli operatori sono le operazioni che appaiono nelle espressioni.

Supponiamo di avere A a1, a2, a3, a4; nell'espressione a1=(a2+a3)*(++a4); compaiono quattro operatori operator=, operator+, operator* e operator++. operator++ ha un solo operando mentre operator=, operator+, operator* ne hanno due.

Gran parte degli operatori si possono ridefinire o come metodi della classe o come funzioni esterne, ma NON le due cose assieme. Quando si ridefiniscono come metodi il primo operando corrisponde all'istanza chiamante e gli altri (se ve ne sono) a parametri del metodo. Quando si ridefiniscono come funzioni esterne alla classe TUTTI gli operandi corrispondono a parametri della funzione.

operator= si può ridefinire SOLO come metodo ovvero

```
1 A& A::operator=(const A&)
```

2.1 Ridefinizione operator<<

```
1 class A{
2     ...
3
4     friend ostream& operator<<(ostream &os, const A& _a);
5 }
6 ostream& operator<<(ostream &os, const A& _a);
```

2.2 Ridefinizione operator+

```
1 class A{
2     ...
3     A operator+(const A&);
4
5     //modo da preferire
6     friend A operator+(const A&, const A&);
7 }
8 A operator+(const A& _x, const A& _y);
```

2.3 Ridefinizione operator-

```
1 class A{
2     ...
3     friend A operator-(const A& _c1, const A& _c2);
4 }
5 A operator-(const A& _c1, const A& _c2);
```

2.4 Ridefinizione operator*

```
1 class A{
2     ...
3     friend A operator*(const A& _c1, const A& _c2);
4 }
5 A operator*(const A& _c1, const A& _c2);
```

2.5 Ridefinizione operator++

```
1  class A{
2      ...
3      A& A::operator++();    //prefisso
4      A A::operator++(int);  //postfisso
5  }
```

2.6 Distinzione sul tipo di ritorno

Ci sono operatori che ritornano il risultato per valore e operatori che ritornano il risultato per referenza. Come nel caso degli incrementi o decrementi prefissi o postfissi la distinzione si ha quando il risultato è utilizzato successivamente nella stessa espressione.

Gli operatori che modificano l'oggetto chiamante ritornando una reference. Esempio: `operator+=` e `operator+` il primo ritornerà `A&` mentre il secondo `A`.

2.7 Parola chiave friend

Quando dentro una classe compare questa parola chiave la riga NON è una dichiarazione di funzione o metodo ma più semplicemente permette l'accesso alle parti private da parte di una funzione o anche classe definita altrove.

3 Ereditarietà

In C++ vi sono tre tipi di ereditarietà:

- public
- private
- protected

Queste sono anche le stesse keywords della visibilità nella classe.

Tabella 1: Ereditarietà

Tipo di Ereditarietà	Casse base	Casse derivata
Public	Public Protected Private	Public Protected Inacc.
Protected	Public Protected Private	Protected Protected Inacc.
Private	Public Protected Private	Private Private Inacc.

3.1 Keyword virtual

I metodi possono essere definiti virtual, in questo modo C++ riesce a dichiarare il late binding (risolto run-time) rispetto al early binding (risolto a compilation-time).

Una classe è detta **Puramente virtuale** se ha almeno un metodo virtual, ovvero la sua dichiarazione nella classe è seguita da "=0":

```
1 class A{
2     virtual int metodo()=0;
3     ...
4 };
```

Una classe puramente virtuale NON si può istanziare.

4 Programmazione generica

Si intende la possibilità data da un linguaggio di rappresentare tipi e implementare algoritmi che abbiano un tipo come parametro. Il tipo parametrico viene poi specificato:

- Tempo di compilazione (es. templates in C++)
- Tempo di esecuzione (es generics in Java)

Lo scopo è implementare algoritmi che siano indipendenti dal tipo su cui operano. Idealmente al massimo livello di astrazione possibile.

```
1 #include<string>
2 #include<utility>
3 #include<iostream>
4 using namespace std;
5
6 template <class T>
7 void my_swap(T& f, T& s) {
8     T tmp = f;
9     f = s;
10    s = tmp;
11 }
12 int main(){
13     int a = 3; int b = 4;
14     my_swap<int> (a,b);      //a=4    b=3
15     string s1 = "hello";
16     string s2 = "world";
17     my_swap<string> (s1,s2); //s1=world s2=hello
18     return 0;
19 }
```

4.1 Utilizzo metodo clone:

```
1 class A{
2     public:
3     virtual A& operator=(const A& _a)=0;
4     virtual A* clone() const=0;
5     virtual ~A(){};
6 };
7 ----
8 #include "A.h"
9 class B:public A{
10     int i;
11     public:
12     B(int _i){i=_i;};
13     B& operator=(const A& _b);
14     B& operator=(const B& _b);
15     B* clone() const;
16     friend ostream& operator<<(ostream& os,const B& _b);
17 };
18 ostream& operator<<(ostream& os,const B& _b);
19 ----
20 #include "B.h"
21 B& B::operator=(const A& _b){
```

```

22     A* temp=_b.clone();
23     B b(*(B*)temp));
24     *this=b;
25     return *this;
26 }
27 B& B::operator=(const B& _b){
28     i=_b.i;
29     return *this;
30 }
31 B* B::clone()const{ return new B(*this);};
32 ostream& operator<<(ostream& os,const B& _b){return os<<_b.i;}
33
34 ----
35
36 #include<string>
37 #include<utility>
38 #include<iostream>
39 using namespace std;
40 #include "B.h"
41
42 void my_swap (A &f, A &s ) {
43     A* temp=f.clone();
44     f=s;
45     s=*temp;
46     delete temp;
47 }
48 int main() {
49     B x(33);
50     B y(44);
51     cout << x << y << endl;
52     my_swap(x,y);
53     cout << x << y << endl;
54     return 0;
55 }

```

5 Standard Template Library

La Standard Template Library (STL) è una libreria software per il linguaggio di programmazione C++ che definisce quattro componenti principali: contenitori, iteratori, algoritmi e funtori.

I contenitori si dividono in sequenziali e associativi:

- I contenitori standard sequenziali includono **vector**, **liste deque**;
- I contenitori standard associativi sono invece: **set**, **multiset**, **map** e **multimap** .

6 Iteratori

Oggetto che consente di visitare tutti gli elementi contenuti in un altro oggetto, tipicamente un contenitore, senza doversi preoccupare dei dettagli di una specifica implementazione. Un iteratore è talvolta chiamato cursore, specialmente nel contesto delle basi dati.

```

1 #include <iostream>
2 using namespace std;
3 #include<list>
4 int main(int argc, char *argv[]){
5     list<int> l;
6     l.push_back(2);
7     l.push_front(3);
8     list<int>::iterator it; //c++98
9     for(it=l.begin();it!=l.end();++it)
10         cout<<*it;

```

Contenitore	Descrizione
vector	Array dinamico capace di ridimensionarsi, elementi contigui accesso, inserimento e cancellazione in coda $O(1)$ inserimento in testa, ricerca, cancellazione $O(n)$
list	lista bidirezionale, elementi non contigui in memoria accesso e ricerca mediante <i>iteratore</i> $O(n)$ inserimento e cancellazione contigui
set	insieme ordinato (piccolo-grande) senza duplicati, elementi non contigui il valore degli elementi non può essere modificato ricerca, inserimento e cancellazione $O(\log(n))$ implementazione BST unordered_set nessun ordinamento, no duplicati in media $O(\log(1))$ implementazione hash table
multiset	uguale al set ma consente duplicati unordered_multiset uguale al unordered_set ma con duplicati
map	array associativo <key, value> ordinato rispetto alla chiave generalmente $O(\log(n))$ $O(\log(1))$ se si accede tramite operator[] unordered_map non ordinato, $O(\log(1))$
multimap	come map, ma può contenere più valori uguali

Tabella 2: Contenitori STL

```

11  for(auto a=l.begin();a!=l.end();++a) //c++11 e successivi
12      cout<<*a;
13  for(auto& e:l)
14      e++; //out: 43
15  cout<<l.front() << l.back(); //out: 43
16  l.push_back(2);
17  l.push_front(5);
18  l.pop_back();
19  l.pop_front();
20
21  auto a=l.cbegin();//const_iterator l=74
22  l.push_front(8);l.push_front(9); //l=9874
23  cout<<endl<<*a; //7
24
25  list<int> l2;
26  l2.splice(l2.cbegin(),l,a,l.cend()); //l: 98 and l2: 74
27  l2.swap(l); //l: 74 and l2: 98
28
29  cout<<l.size(); //2
30
31  l.reverse(); //47
32  l.sort(); //47 (773344->334477)
33  l.unique(); //47 (347)
34  l2.sort() //89
35  l.merge(l2); //l: 4789 l2:
36
37  //--reverse iterator--
38
39  list<int>::reverse_iterator rit;
40  for(rit=l.rbegin();rit!=l.rend();++rit)
41      cout<<*rit; //9874
42  for(auto a=l.rbegin();a!=l.rend();++a)
43      cout<<*a;
44
45  l.resize(3); //l: 478
46  l.remove(7); //l: 48
47  l.erase(l.begin()); //l: 8
48  a=l.emplace(l.cbegin(),11); //l: 118

```

```

49     l.push_front(13)           //l: 13118
50
51     l2.assign(a,l.cend());      //l: 13118 l: 118
52
53     list<int> l3(1);           //l3: 13118
54     l=l2;                      //l: 118    l2:118
55
56     l.clear();                 //l:
57     cout<<endl<<l.max_size();  //768614336404564650
58 }

```

7 set & map

Sono ordinati e usano operator<, non hanno il concetto di front e back, ma si usa insert. Per utilizzare l'inserimento in map si deve utilizzare un nuovo operatore pair<T,T>

```

1  #include <iostream>
2  using namespace std;
3  #include<set>
4  #include<map>
5
6  int main(int argc, char *argv[]){
7      set<A> s;
8      s.insert(A()); // funziona solo se definito A::operator<
9      map<A,int> m;
10     m.insert(pair<A,int> (A(),2)); // funziona solo se definito A::operator<
11     map<int,A> ma;
12     ma.insert(pair<int,A> (3,A())); //funziona, operator< definito per int
13     ma[3]=A(); //equivalentemente per il map e' ridefinito operator[]
14 }

```

8 multiset & multimap

Sono simili ai loro genitori: permettono di avere più elementi con lo stesso valore. Non può essere usato operator[].

9 STL <algorithm>

Alcune funzioni della libreria algorithm, per l'elenco completo si veda il seguente link

- all_of: Test condition on all elements in range
- any_of: Test if any element in range fulfills condition
- none_of: Test if no elements fulfill condition
- for_each: Apply function to range
- find: Find value in range
- find_if: Find element in range
- find_if_not: Find element in range (negative condition)
- count_if: Return number of elements in range satisfying condition
- mismatch: Return first position where two ranges differ
- equal: Test whether the elements in two ranges are equal
- is_permutation: Test whether range is permutation of another

- `copy`: Copy range of elements
- `replace`: Replace value in range (function template)
- `replace_if`: Replace values in range
- `remove`: Remove value from range
- `random_shuffle`: Randomly rearrange elements in range
- `reverse`: Reverse range
- `rotate`: Rotate left the elements in range
- `sort`: Sort elements in range

10 Puntatori e Funzioni

Variabili che contengono indirizzi a funzioni. Questi servono quando il programma deve scegliere quale funzione chiamare fra diverse possibili, e la scelta non è definita a priori ma dipende dai dati del programma stesso.

Questo processo si chiama *late binding*: gli indirizzi delle funzioni da chiamare non vengono risolti al momento della compilazione, come avviene normalmente (*early binding*) ma al momento dell'esecuzione.

```

1  int a(double, int){...}
2  int b(double, int){...}
3  int f(int i, int j){return i*j;}
4  int f2(int i, int j){return i+j;}
5  double media_generale(int i, int j, int (*pf)(int, int) ){
6      return double(pf(i, j))/2;
7  }
8  main(){
9      int (*pf)(int, int);
10     int (*pfunz)(double, int);
11     ...
12     if(i%2==0){
13         pfunz=a;
14     }else
15         pfunz=b;
16     }
17     cout<<pfunz(2.0, 3); //or cout<<(*pfunz)(2.0, 3);
18
19     pf=f;
20     cout<<pf(2, 3)<<endl; // or (*pf)(2, 3); //6
21     cout<<media_generale(2, 3, f)<<endl; //3
22     cout<<media_generale(2, 3, f2)<<endl; //2.5
23 }
```

11 Eccezioni

Gli errori di runtime sono quegli errori che NON possono essere rilevati in fase di compilazione, perché si manifestano solo durante la fase di esecuzione del programma e solo in alcune particolari circostanze, ossia ai veri casi di “eventi eccezionali”.

Alcuni esempi:

- divisione per 0;
- pathname non valido o che fa riferimento ad una periferica che in un certo momento risulta scollegata;
- overflow su un tipo di dato

- operazione di casting non valido

Si tratta di errori pericolosi perché se non gestiti, possono generare anomalie di funzionamento che possono determinare persino il blocco inaspettato del programma.

Un'eccezione non necessariamente deve essere gestita all'interno della stessa funzione in cui viene sollevata. Questo è reso possibile dal meccanismo proprio delle eccezioni, il quale prevede che un'eccezione sollevata in una funzione e che non venga in essa gestita, sia propagata all'indietro verso il chiamante, si dice risalendo lo stack delle chiamate.

11.1 throw

Per gestire questi errori i linguaggi OOP implementano il meccanismo delle eccezioni, il quale consente di sollevare, **throw**, un'eccezione quando si verifica un errore a runtime. Questo meccanismo si traduce nella creazione di un oggetto appartiene ad una classe particolare, dipendente dallo specifico errore di runtime che si è verificato.

11.2 try-cath

Per "intercettare" l'errore in C++ viene fornito il costrutto **try-cath** il quale permette di sospendere il programma nel punto dove si è verificato l'errore per saltare al pezzo di codice che gestisce l'errore.

```
1 try{
2     int x, y;
3     //insert from keyboard
4     if(y == 0) throw "Division by zero!";    //or division(x,y);
5 }catch(const char* msg){
6     cerr<<msg<< endl;
7 }catch(){ }
8 ...
```

try: blocco in cui si possono inserire le istruzioni che si vuole tenere “sotto controllo” perché durante l'esecuzione potrebbero generare degli errori a runtime.

catch: blocco in grado di riconoscere e “catturare” un certo tipo di eccezione e che contiene le istruzioni per gestire l'errore di runtime che l'ha generata.

Quando nel blocco **try** si genera un'eccezione si salta ai suoi blocchi **catch** associati, partendo dal primo a scendere (dall'eccezione più specifica a quella più generale). Alla sua uscita il flusso del programma prosegue oltre l'ultimo blocco **catch**.

11.3 stdexcept

Questo header definisce un insieme di eccezioni standard che sia la libreria che i programmi possono utilizzare per segnalare errori comuni. Includere nell'intestazione: `#include <stdexcept>`.

- Logic errors
 - `logic_error`
 - `domain_error`
 - `invalid_argument`
 - `length_error`
 - `out_of_range`
- Runtime errors
 - `runtime_error`
 - `range_error`
 - `overflow_error`
 - `underflow_error`

```

1 #include <stdexcept>
2 using namespace std;
3
4 int divideINT(int a, int b) {
5     if (a < 0 || b < 0) {
6         throw invalid_argument( "received negative value" );
7     }
8     return a/b;
9 }
10 void test(){
11     int x=1, y=0;
12     try {
13         cout << a << "/" << b << "=" << divideINT(a, b) << endl;
14     } catch( const invalid_argument& e ) {
15         cout << e.what(); << endl //or cout << "a or b negative" << endl;    //cout << e
16         << endl; //bind ERROR
17     } catch(...){ //otherwise
18         cout << "other wrong error";
19     }
20 }
21 int main(){ test(); }

```

12 Multithreading in C++

Dal C++ 11 si ha la classe `thread`, la quale abilita alla scrittura di programmi che possono eseguire parti del codice in parallelo e non più in maniera sequenziale.

La classe `thread` ha le seguenti funzioni:

- `detach`
- `get_id`
- `join`
- `joinable`
- `join`
- `joinable`
- `native_handle`
- `operator=` (move semantics)
- `hardware_concurrency()`

```

1 #include <iostream>
2 #include <thread>
3 using namespace std;
4
5 int a(){ /*do something...*/ }
6 int b(int count){ /*do something...*/ }
7
8 int main(){
9     unsigned int c = thread::hardware_concurrency();
10    thread t1 (a);
11    thread t2;
12    t2 = thread(b,0);
13
14    t1.join();
15    t2.join();
16 }

```

12.1 Data race

Si verifica quando due o più threads in un singolo processo accedono simultaneamente alla stessa area di memoria, almeno un accesso è eseguito in scrittura e non vi sono blocchi per l'accesso esclusivo. Questo significa che il valore da un momento all'altro può essere cambiato ed alterare il funzionamento del programma.

Proprio per questo motivo in C++ vengono introdotti due blocchi: `atomic` e `mutex`.

- `atomic` rende atomiche le operazioni su una certa variabile;
- `mutex` Definisce delle sezioni critiche come mutuamente esclusive nel codice;

```
1 //first resolution with atomic
2 #include <iostream>
3 #include <thread>
4 #include <atomic>
5 using namespace std;
6
7 static std::atomic<int> count(0);
8 //static int count=0;
9 void inc_thread(){
10     for(int i=0;i<10000;i++) count++;
11 }
12 void dec_thread(){
13     for (int i=0;i<10000;i++) count--;
14 }
15 main(){
16     std::thread inc(inc_thread);
17     std::thread dec(dec_thread);
18     inc.join();
19     dec.join();
20     std::cout<<"counter"<<count;
21     return 0;
22 }
```

```
1 //second resolution with mutex
2 #include <iostream>
3 #include <thread>
4 #include <mutex>
5 using namespace std;
6
7 mutex myMutex;
8 static int condivisa=0;
9 void inc_thread(){
10     for(int i=0;i<10000;i++){
11         myMutex.lock(); condivisa++; myMutex.unlock();
12     }
13 void dec_thread(){
14     for (int i=0;i<10000;i++){
15         myMutex.lock(); condivisa--; myMutex.unlock();
16     }
17 int main (){
18     std::thread inc(inc_thread);
19     std::thread dec(dec_thread);
20     inc.join();
21     dec.join();
22 }
```

12.2 Problemi della concorrenza: deadlock e starvation

Deadlock Situazioni in cui uno o più processi/thread si bloccano:

- Omettere l'unlock un mutex
- Terminazione inattesa di una funzione (lancio di eccezioni), soluzione: `lock_guard<mutex>`

- Funzioni annidate che chiamano lo stesso mutex, soluzione: `recursive_mutex`
- Ordine di locking di diversi mutex , soluzione: `ock(mutex, mutex)`

Starvation La politica di accesso impedisce ad un processo di accedere alla risorsa

Esistono nella letteratura altre primitive usando per gestire la concorrenza come semafori e monitor ma non sono forniti dallo standard di C++ che preferisce fornire primitive di livello più basso.

13 Riferimenti: lvalues & rvalues

Un riferimento, analogamente a un puntatore, archivia l'indirizzo di un oggetto che si trova in un'altra posizione in memoria. A differenza di un puntatore, dopo l'inizializzazione non è possibile impostare un riferimento in modo che indichi un oggetto diverso o che sia impostato su Null. Esistono due tipi di riferimenti e operatori:

- L'operatore `&` indica: riferimenti **lvalue** che fanno riferimento a una variabile denominata
- L'operatore `&&` indica: riferimenti **rvalue**, o un riferimento universale (**rvalue** o **lvalue**) a seconda del contesto, che fanno riferimento a un oggetto temporaneo.

```

1 #include <iostream>
2 using namespace std;
3
4 int main(){
5     Person myFriend;    //struct with char* Name and Short Age
6
7     Person& rFriend = myFriend;
8
9     myFriend.Name = "Bill";
10    rFriend.Age = 40;
11
12    cout << rFriend.Name << " is " << myFriend.Age << endl; //Bill is 40
13 }
```

13.1 lvalues: &

È possibile considerare un riferimento lvalue come nome alternativo per un oggetto. Un riferimento deve essere inizializzato e non può essere modificato.

13.2 Rvalue: &&

I riferimenti Rvalue supportano l'implementazione della semantica di spostamento, che può aumentare significativamente le prestazioni delle applicazioni. La semantica di spostamento, `move()`, consente di scrivere codice per il trasferimento delle risorse (ad esempio memoria allocata in modo dinamico) da un oggetto a un altro. L'idea è che la proprietà di dati allocati nella memoria dinamica venga trasferita senza dover essere copiata. Lo scopo è aumentare l'efficienza del programma copiando meno dati possibili.

Per implementare la move semantics si fornisce:

- costruttore di spostamento
- operatore di assegnazione di spostamento `operator=` alla classe
- distruttore

```

1 #include <iostream>
2 using namespace std;
3
4 int main(){
5     int& x=6; //non compila
6 }
```

```

6  const int& x=6; //compila
7  //Dal c++11
8  int&& i=7; // rvalue reference, Permette di fare riferimento ad un rvalue e di
    modificarlo
9  }

```

le reference usuali `int &` sono lvalue references.

Per comprendere meglio la semantica di spostamento, si consideri l'esempio dell'inserimento di un elemento in un oggetto `vector`. Se viene superata la capacità dell'oggetto, l'oggetto `vector` deve riallocare memoria sufficiente per i relativi elementi e quindi copiare ogni elemento in un'altra posizione di memoria per liberare spazio per l'elemento inserito. Quando un'operazione di inserimento copia un elemento, crea prima di tutto un nuovo elemento. Chiama quindi il costruttore di copia per copiare i dati dall'elemento precedente al nuovo elemento. Infine, elimina definitivamente l'elemento precedente. La semantica di spostamento consente di spostare gli oggetti direttamente senza dover eseguire operazioni di allocazione di memoria e copia costose.

13.3 Value Categories

- **glvalue**, general lvalue espressione la cui valutazione determina l'identità di un oggetto, di un campo di bit o di una funzione.
- **prvalue**, pure rvalue espressione la cui valutazione inizializza un oggetto o un campo di bit oppure calcola il valore dell'operando di un operatore
- **xvalue**, eXpiring value glvalue che denota un oggetto o un campo di bit le cui risorse possono essere riutilizzate, in genere perchè vicino a fine vita
- **lvalue** è un glvalue che non è un valore xvalue.
- **glvalue** è un prvalue o un valore xvalue.

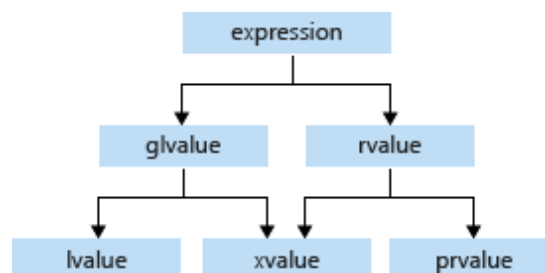


Figura 1: Value CategoriesValue

Un lvalue ha un indirizzo a cui il programma può accedere, ad esempio nomi di variabili.

Un'espressione prvalue non ha un indirizzo accessibile dal programma. Esempi di espressioni prvalue includono valori letterali, chiamate di funzione che restituiscono un tipo non di riferimento e oggetti temporanei creati durante la valutazione dell'espressione, ma accessibili solo dal compilatore.

Un'espressione xvalue ha un indirizzo che non è più accessibile dal programma, ma può essere usato per inizializzare un riferimento rvalue, che fornisce l'accesso all'espressione.

13.4 std::move

Funzione template che restituisce una rvalue reference al suo argomento. Il suo scopo è "cannibalizzare" l'oggetto per trasferirlo ad un altro riferimento

```

1  void swap_copy(T& a, T& b){
2      T& temp(a);
3      a = b;
4      b = temp;
5  }

1  void swap_move(T& a, T& b){
2      T& temp(move(a));
3      a = move(b);
4      b = move(temp);
5  }

```

13.5 Default/delete

È possibile specificare con default and delete quali di quali metodi adottare la versione di default e quali non consentire.

```
1 class A{
2     int i;
3     public:
4     A(const A& _a)=default;
5     A(A&& _a):i=0{
6         i = _a.i;
7         _a.i=0;        //if you want
8     }
9     A& operator=(const A& _a)=default; //a=(b=c)
10    A& operator=(A&& _a){
11        i=_a.i;
12        _a.i=0;
13        return *this;
14    }
15 };
```

14 Smart pointers

Puntatori intelligenti che provvedono in automatico alla cancellazione della memoria. Non fanno parte delle caratteristiche basi del C++ e proprio per questo vengono forniti attraverso le librerie tra cui quella standard: `#include <iostream>`.

Dopo essere stato inizializzato, il puntatore intelligente detiene la proprietà del puntatore non elaborato. Ciò significa che il puntatore intelligente è responsabile dell'eliminazione della memoria specificata dal puntatore non elaborato. Il distruttore del puntatore intelligente contiene la chiamata per l'eliminazione e poiché il puntatore intelligente viene dichiarato nello stack, il relativo distruttore viene richiamato quando il puntatore intelligente esce dall'ambito, anche se viene generata un'eccezione in un punto precedente dello stack.

Accedere al puntatore incapsulato utilizzando gli operatori del puntatore noti, `->` e `*`, di cui verrà eseguito l'overload per restituire il puntatore non elaborato incapsulato.

Nel linguaggio C++ moderno, i puntatori non elaborati vengono utilizzati esclusivamente in blocchi di codice piccoli con ambito limitato, nei cicli o nelle funzioni di supporto per le quali le prestazioni sono importanti e non può crearsi confusione circa la proprietà.

La differenza, rispetto ad altri linguaggi, consiste nel fatto che nessun Garbage Collector separato viene eseguito in background. La memoria viene gestita tramite le regole di ambito C++ standard in modo che l'ambiente di runtime risulti più veloce e più efficiente.

IMPORTANTE: Creare sempre puntatori intelligenti su una riga di codice separata e mai in un elenco di parametri, in modo da evitare anche le più piccole perdite di risorse dovute a alcune regole di allocazione dell'elenco di parametri.

```
1 #include <iostream>
2 #include <memory>
3
4 void UseRawPointer(){
5     // Using a raw pointer -- not recommended.
6     Song* pSong = new Song(L"Nothing on You", L"Bruno Mars");
7
8     // Use pSong...
9
10    // Don't forget to delete!
11    delete pSong;
12 }
13
14 void UseSmartPointer(){
15     // Declare a smart pointer on stack and pass it the raw pointer.
```

```

16     unique_ptr<Song> song2(new Song(L"Nothing on You", L"Bruno Mars"));
17
18     unique_ptr<int> p;
19     p.reset(new int(54));
20
21     // Use song2...
22     wstring s = song2->duration_;
23     //...
24
25 } // song2 is deleted automatically here.

```

14.1 unique_ptr

Classe template che permette di gestire una risorsa nella memoria dinamica garantendone l'ownership. Può essere spostato a un nuovo proprietario, ma non copiato o condiviso. Sostituisce `auto_ptr`, che è deprecato.

14.2 shared_ptr

Puntatore intelligente con conteggio dei riferimenti, per accedere al conteggio viene fornito il metodo `use_count()`. Possiede più ownership, utilizzato ad esempio quando si restituisce una copia di un puntatore da un contenitore, ma si desidera conservare l'originale.

Il puntatore non elaborato non viene eliminato finché tutti i proprietari di `shared_ptr` non sono usciti dall'ambito o non hanno ceduto in altro modo la proprietà.

```

1 class A{
2     shared_ptr<int> p;
3     int owner(){
4         return p.use_count();
5     }
6 };

```

14.3 weak_ptr

Puntatore intelligente per casi speciali da utilizzare insieme a `shared_ptr`. `weak_ptr` fornisce l'accesso a un oggetto di proprietà di una o più istanze di `shared_ptr`, ma non partecipa al conteggio dei riferimenti. Utilizzarlo quando si desidera osservare un oggetto, ma non è necessario che rimanga attivo.

Per accedere al contenuto e' necessario copiarlo in uno `shared_ptr` usando un metodo `lock()`.

```

1 class A{
2     weak_ptr<int> wp;
3
4     int get_wp(){
5         shared_ptr<int> sp=wp.lock();
6         return *sp;
7     }
8 };

```

14.4 Metodi utili

```

1 #include <iostream>
2 #include <memory>
3 using namespace std;
4
5 class A{
6     unique_ptr<int> wp;
7     unique_ptr<Data> wp_data;
8
9     p.reset(new int(54));
10    cout << *p;
11
12    p.reset()    // Free the memory before we exit function block.
13

```

```

14 ---
15
16 wp_data -> doSomething(); //Call a method on the object
17 Data* data = wp_data.get(); //Pass raw pointer get()
18
19 ---
20
21 shared_ptr<int> p;
22 int numer_owners = p.use_count();
23 };

```

15 Bitset

Template class della standard template library realizza un'array di bit, e l'implementazione permette di rappresentare i bit come bit fisici. Alcune funzioni della libreria algorithm, per l'elenco completo si veda il seguente link

- `operator[]`: Access bit
- `count`: Count bits set
- `size`: Return size
- `test`: Return bit value
- `any`: Test if any bit is set
- `none`: Test if no bit is set
- `all`: Test if all bits are set
- `set`: Set bits
- `reset`: Reset bits
- `flip`: Flip bits
- `to_string`: Convert to string
- `to_ulong`: Convert to unsigned long integer
- `to_ullong`: Convert to unsigned long long

```

1 #include <iostream>
2 #include <bitset>
3 #include <string>
4 #include <cstdint> // std::size_t
5 using namespace std;
6 int main (){
7     bitset<4> foo;
8     cout << foo.set() << '\n'; // 1111
9     cout << foo.set(2,0) << '\n'; // 1011
10    cout << foo.set(2) << '\n'; // 1111
11    cout << foo << " as an integer is: " << foo.to_ulong() << '\n';
12    //1111 as an integer is: 15
13
14    bitset<4> fo (string("1011"));
15    cout << fo.reset(1) << '\n'; // 1001
16    cout << fo.reset() << '\n'; // 0000
17
18    bitset<4> f (string("0001"));
19    cout << f.flip(2) << '\n'; // 0101
20    cout << f.flip() << '\n'; // 1010
21

```


[illegible]

16 Espressioni lambda

Le espressioni lambda si possono pensare come funzioni senza nome, esse sono definite la dove vengono anche usate e il loro punto essenziale è la cattura di variabili dal contesto in cui appaiono. Queste espressioni lambda vengono spesso utilizzate come argomento per le funzioni che accettano un oggetto chiamabile.

Ciò può essere più semplice della creazione di una funzione con nome, che verrebbe utilizzata solo quando passata come argomento. In tali casi, le espressioni lambda sono generalmente preferite perché consentono di definire gli oggetti funzione in linea.

In pratica una espressione lambda è un altro modo di definire in C++ un'entità invocabile che può essere usata in maniera diretta o passata come argomento ad un'altra funzione, ma la cui applicazione è di scarsa utilità al di fuori del contesto di definizione. • In questi casi, la definizione di una espressione lambda risulta meno prolissa della definizione di una funzione o un funtore ad hoc ed ha il grande vantaggio di essere localizzata nel contesto d'uso.

```
[ captures ] ( <params> ) <mutable> < -> > <return_type> { <body_of_function> }
```

La cattura specifica il modo di cattura per riferimento [&] o per valore/copia [=]. Nella clausola di cattura si può definire la cattura di default inserendo come primo paramentro [&] oppure [=], facendo così però non si può più inserire [*identifier*] oppure [= *identifier*], perchè questi sono già stati catturati con il metodo di default. L'uso del qualificare **mutable** in questo contesto rende infatti possibile alterare il valore delle variabili catturate per valore nel corpo di istruzioni di una espressione lambda. In pratica, le alterazioni prodotte dal corpo della espressione lambda persistono tra un'esecuzione e l'altra, ma non hanno effetti nel contesto esterno.

```

1 struct S { void f(int i); };
2
3 void S::f(int i) {
4     [&, i]{};           // OK
5     [&i]{};             // OK
6     [&, &i]{};          // ERROR: i preceded by & when & is the default
7     [=, this]{};         // ERROR: this when = is the default
8     [=, *this]{ };       // OK: captures this by value. See below.
9     [i, i]{};            // ERROR: i repeated
10 }

```

La cattura per valore, come discusso in precedenza, consente di catturare per copia una variabile dal contesto esterno alla espressione. Per default, tale copia è accessibile all'interno del corpo di istruzioni in sola lettura, ma non è modificabile. • Ciò consente di invocare la medesima espressione lambda più volte, senza il rischio di alterare i valori catturati in origine dal contesto esterno tra un'esecuzione e l'altra. • La cattura per riferimento, invece, consente di alterare liberamente le variabili catturate dall'esterno. Come conseguenza di ciò, a seguito dell'esecuzione dell'espressione lambda, il contesto di invocazione risulta alterato.

```

1 #include <iostream>
2 #include <vector>
3 #include <numeric>           //for iota
4 #include <list>
5 #include <algorithm>
6 using namespace std;
7
8 class Amico{
9     private:
10     string nome;
11     int eta;
12     public:
13     //operator ++ e -
14 };
15
16 void incEta(list<vector<Amico>>& gruppi){
17     auto modify = [&gruppi]() {
18         for (auto& gr : gruppi) {
19             for (auto& am : gr) ++am;
20         }
21     };
22     modify();
23 }
24
25 int main(){
26     auto somma = [](int x, int y) { return x + y; };
27     //cout << "somma=" << somma << endl; //errore
28     cout << "somma=" << somma(5, 8) << endl;
29
30     vector<int> vec(10);
31     iota(v.begin(), v.end(), 0);
32     for(auto item: vec){ cout << "["<<item<<"]"; }
33     cout << endl;
34     // Trova un numero inferiore a un dato input
35     int threshold = 10;
36     auto it = find_if( vec.begin(), vec.end(),
37     [threshold](int value) { return value < threshold; } );
38
39
40     int num = 3;
41     // trasforma il contenuto di vec moltiplicandolo per num
42     transform(vec.begin(), vec.end(), vec.begin(),
43     [num] (const int& e) {
44         return e * num;
45     } );
46
47     // rimuove tutti gli elementi minori di soglia
48     int soglia = 10;
49     //[soglia](int n) { return n < soglia; }
50     vec.erase( remove_if(vec.begin(), vec.end(),
51     [soglia](int n) {
52         return n < soglia;
53     } ),
54     vec.end() );
55
56
57
58
59     list<int> mylist{ 1, -2, 3, 4, -5, 8, 6};
60     //VERSIONE 1:: rimuove tutti gli elementi pari con una
61     espressione lambda
62     auto fremove = [&mylist] () {
63     for(auto it = mylist.begin(); it != mylist.end(); ){
64         if((*it)%2==0){
65             auto it2remove = it;
66             it++;
67             mylist.erase(it2remove);

```

```

68         } else {
69             it++;
70         }
71     };
72     fremove();
73
74
75
76     //metodo remove_if delle liste
77     void rimuoviPari1 (list<int>& l) {
78         l.remove_if( [](int i) {return i%2==0;} );
79     }
80     //funzione remove_if di algorithm
81     void rimuoviPari2 (list<int>& l) {
82         remove_if(l.begin(), l.end(),
83             [](int i) ->bool { return i%2==0; } );
84     }
85     rimuoviPari1(mylist);
86     //rimuoviPari2(mylist);
87
88
89     //VERSIONE 3:: rimuove tutti gli elementi pari con una espressione lambda
90     auto rimuovi=[&mylist]()mutable{mylist.remove_if (pari); };
91     //forma alternativa
92     //auto rimuovi = [&]()mutable{mylist.remove_if (pari); };
93     rimuovi();
94
95
96
97     return 0;
98 }

```